

 Claude Platform Docs

MessagesManaged AgentsAdminResources ▾

{ } API r

Q Search



Thinking

Tools

Overview

How tool use works

Tutorial: Build a tool-using agent

Define tools

Handle tool calls

Parallel tool use

Tool Runner (SDK)

Strict tool use

Server tools

Web search tool

Web fetch tool

Code execution tool

Advisor tool

Tool search tool

Memory tool

 **Béatrice**
Admin · Béatrice's I... ▾

Messages > Tools

How tool use works

 Copy page ▾

Understand the tool use loop, where tools execute, and when to use tools instead of prose.

This page explains the concepts behind tool use: where tools run, how the agentic loop works, and when tool use is the right approach. For hands-on guidance, start with the [Build a tool-using agent](#) tutorial or the [Define tools](#) guide.

The tool-use contract

Tool use is a contract between your application and the model. You specify what operations are available and what shape their inputs and outputs take; Claude determines when and how to call them. The model never executes anything on its own. It emits a structured request, your code (or Anthropic's servers) runs the operation, and the result flows back into the conversation.

This contract makes the model behave less like a text generator and more like a function you call. Engineers with classical API experience can integrate tool use the same way they would any other typed interface: define the schema, handle the callback, return a result. The difference is that the caller on the other side is a language model choosing which function to call based on the conversation.

Where tools run

The primary axis along which tools differ is where the code executes. Every tool falls into one of three buckets, and the bucket determines what your application is responsible for.

Ask Docs 

User-defined tools (client-executed)

You write the schema, you execute the code, you return the results. This is the most common case: the vast majority of tool-use traffic is user-defined tools calling into application-specific logic.

When Claude calls one of your tools, the API response contains a `tool_use` block with the tool name and a JSON object of arguments. Your application extracts those arguments, runs the operation (a database query, an HTTP call, a file write, whatever the tool does), and sends the output back in a `tool_result` block on the next request. Claude never sees your implementation; it only sees the schema you provided and the result you returned.

Anthropic-schema tools (client-executed)

For a handful of common operations (managing scratchpad memory, running shell commands, editing files, controlling a desktop or a browser), Anthropic publishes the tool schema and your application handles execution. The tools in this category are memory , bash , text_editor , computer , and browser .

The execution model is identical to user-defined tools: the response contains a `tool_use` block, your code runs the operation, and you send back a `tool_result` . The reason to use an Anthropic-schema tool instead of defining your own equivalent is that these schemas are trained-in. Claude has been optimized on thousands of successful trajectories that use these exact tool signatures, so it calls them more reliably and recovers from errors more gracefully than it would with a custom tool that does the same thing. The schema is the interface the model already expects.

Server-executed tools

For web_search , web_fetch ,

`code_execution` , and `tool_search` , Anthropic runs the code. You enable the tool in your request and the server handles everything else. You never construct a `tool_result` block for these tools. When a turn calls only `server_tools`, the server-side loop executes the operation and feeds the output back to the model before the response reaches you, unless the loop stops before it finishes, most often because it pauses.

The response you receive contains `server_tool_use` blocks showing what ran and what came back. In the common case, execution is already complete by the time you see them, and your application's job is to enable the tool and read the final answer rather than to participate in the execution loop; the main exceptions are a paused loop (`pause_turn`) and a turn that also calls a client tool.

The agentic loop (client tools)

Client-executed tools (both user-defined and Anthropic-schema) require your application to drive a loop. The model can't run your code, so every tool call is a round trip: the model asks, you execute, you report back, the model continues.

The canonical shape is a `while` loop keyed on `stop_reason` :

1. Send a request with your `tools` array and the user message.
2. Claude responds with `stop_reason`: `"tool_use"` and one or more `tool_use` blocks.
3. Execute each tool. Format the outputs as `tool_result` blocks.
4. Send a new request containing the original messages, the assistant's response, and a user message with the `tool_result` blocks.
5. Repeat from step 2 while `stop_reason` is `"tool_use"` .

In practice this reads as: while `stop_reason == "tool_use"` , execute the tools and

continue the conversation. The loop exits on any other stop reason ("end_turn" , "max_tokens" , "stop_sequence" , or "refusal"), which means Claude has either produced a final answer or stopped for another reason that your application should handle.

For the mechanics of building requests, handling parallel tool calls, and formatting results, see [Handle tool calls](#).

The server-side loop

Server-executed tools run their own loop inside Anthropic's infrastructure. A single request from your application might trigger several web searches or code executions before a response comes back. The model searches, reads results, determines whether to search again, and iterates until it has what it needs, all without your application participating.

This internal loop has an iteration limit. If the model is still iterating when it hits the cap, the response comes back with `stop_reason: "pause_turn"` instead of `"end_turn"` . A paused turn means the work isn't finished; re-send the conversation (including the paused response) to let the model continue where it left off. See [Server tools](#) for the continuation pattern.

The loop also hands control back to you before a server tool runs if Claude calls that server tool and a client tool in the same group of parallel tool calls. The response then comes back with `stop_reason: "tool_use"` and a `server_tool_use` block that has no result block yet; the API runs it after you return the client tool results. See [Stop reasons and fallback](#) for the exact contract.

When to use tools (and when not to)

Tool use fits when the task requires something the model can't do from text alone:

- **Actions with side effects.** Sending an email, writing a file, updating a record.

The model can describe these actions, but only a tool can perform them.

- **Fresh or external data.** Current prices, today's weather, the contents of a database. Anything outside the training data or specific to your system needs a tool to fetch it.
- **Structured, guaranteed-shape outputs.** When you need a JSON object with specific fields rather than prose that happens to contain the information, a tool schema enforces the shape.
- **Calling into existing systems.** Databases, internal APIs, filesystems. Tool use is the bridge between natural-language requests and the systems that fulfill them.

A clear sign that you should be using tools: if you're writing a regex to extract a decision from model output, that decision should have been a tool call. Parsing free-form text to recover structured intent is a sign the structure belongs in the schema.

Tool use doesn't fit when:

- The model can answer from training alone. Summarization, translation, and general-knowledge questions don't need a tool round trip.
- The interaction is one-shot Q&A with no side effects. If there's nothing to execute, there's nothing for a tool to do.
- Tool-calling latency would dominate a trivial response. Every tool call is at least one extra round trip; for lightweight tasks the overhead can exceed the work.

Choosing between approaches

Approach	When to use it	What to expect	Learn more
User-defined client	Custom business logic, internal	You handle execution and the	Define

tools	APIs, proprietary data	agentic loop	<u>tools</u>
Anthropic-schema client tools	Standard dev operations (bash, file editing, desktop and browser control)	You handle execution; Claude calls the tool reliably because the schema is trained-in	<u>Tool reference</u>
Server-executed tools	Web search, code sandbox, web fetch	Anthropic handles execution; you read the results instead of producing them	<u>Server tools</u>

Next steps

Tutorial:
Build a tool-using agent

Build an agent...

Define tools

Schema specifi...

Tool reference

Director y of...

Was this page helpful?



Claude Platform Docs



Solutions
AI agents
Code modernization
Coding Research
Customer support
Financial services
Government
Higher education
K-12 teachers
Life sciences
Partners
Claude on AWS
Claude on Google Cloud
Compliance
Anthropic
Security
and Events
powered by
Transparency
Claude policy
Service partners
Startups
Commercial
Help and security
Courses
Availability
Status
Support Connectors
Discord
News Customer stories
Responsible
Scaling Engineering
Policy at
Anthropic
Privacy
policy
compliance
Responsible disclosure
Terms of service
Consumer Usage policy